

实验五：基于 YOLOv5 的 PASCAL VOC 目标检测微调与评估

小组成员：朱立景、徐康晴、吕正大

实验代码仓库：https://github.com/xinhanzhongjiu/UCAS_DeepLearning_2026

一、概述

1.1 任务背景

本实验旨在利用深度学习目标检测模型 **YOLOv5s**，在公开数据集 **PASCAL VOC** 上进行迁移学习（微调），使在 COCO 数据集上预训练的通用检测器适配 VOC 的 20 类目标检测任务，并系统评估微调前后模型在精度、召回率、F1 分数及 mAP 等指标上的变化。

1.2 数据集

采用 **PASCAL VOC 2007 + 2012** 合并数据集，共 20 个类别（人、车、动物、家具等日常物体）。训练集约 **16,551** 张图像，测试集 **4,952** 张（`test2007`），标注格式转换为 YOLO 所需的归一化边界框。

1.3 解决方案概要

模块	方案
基础模型	YOLOv5s (COCO 预训练, <code>yolov5s.pt</code>)
实验 A (Baseline)	默认低增强超参 <code>hyp.scratch-low.yaml</code> , <code>lr=0.01</code>
实验 B (调优)	自定义 <code>hyp.voc_finetune.yaml</code> , 低学习率 + 部分冻结 backbone + mixup
训练策略	50 epoch, batch=32, 输入 640×640, SGD 优化器
评估	<code>val.py</code> 计算 P/R/F1/mAP; <code>detect.py</code> 批量可视化
分析工具	自研 <code>analyze_results.py</code> 对比训练曲线与指标

1.4 实验环境

- 框架：PyTorch 2.6 + Ultralytics YOLOv5 v7.0
- 环境：Conda `yolo` (python=3.12 torch=2.6.0+cu124) , NVIDIA A800 GPU
- 代码目录：`exp5/yolov5/`

二、解决方案

2.1 网络结构设计

本实验采用 **YOLOv5s** 作为检测骨干，属于单阶段（One-Stage）Anchor-Based 检测器。网络由 **Backbone + Neck (FPN/PAN) + Head (Detect)** 三部分组成，在三个尺度（P3/8、P4/16、P5/32）上进行多尺度预测。

结构要点：

- **Backbone**：CSPDarknet 风格，含 Conv、C3（Cross Stage Partial）、SPPF 模块，逐层下采样提取特征

- **Neck:** FPN + PAN 结构，通过上采样与 Concat 融合浅层细节与深层语义
- **Head:** Detect 层，每个尺度输出 `(x, y, w, h, obj, cls×20)` 共 25 维（VOC 20 类）

模型定义见 `yolov5/models/yolov5s.yaml`：

```

1  # Backbone: P1→P5 五级特征金字塔
2  backbone:
3    [[-1, 1, Conv, [64, 6, 2, 2]], # 0-P1/2
4     [-1, 1, Conv, [128, 3, 2]], # 1-P2/4
5     [-1, 3, C3, [128]],
6     [-1, 1, Conv, [256, 3, 2]], # 3-P3/8
7     ...
8     [-1, 1, SPPF, [1024, 5]], # 9
9   ]
10
11 # Head: FPN+PAN 三尺度检测
12 head:
13   [...]
14   [[17, 20, 23], 1, Detect, [nc, anchors]], # P3, P4, P5
15   ]

```

微调时，检测头类别数由 COCO 的 80 类自动调整为 VOC 的 20 类（`train.py` 根据 `data/VOC.yaml` 中的 `names` 字段重建输出层）。

参数量：约 7.06M（VOC 微调后），推理约 15.9 GFLOPs（640×640 输入）。

2.2 损失函数设计

YOLOv5 的总损失由三项加权组成：

$$\mathcal{L} = \lambda_{box}\mathcal{L}_{box} + \lambda_{obj}\mathcal{L}_{obj} + \lambda_{cls}\mathcal{L}_{cls}$$

损失项	含义	实现
$\mathcal{L}_{box}$	边界框回归损失	$1 - IoU$ （GloU/DIoU 形式）
$\mathcal{L}_{obj}$	目标置信度损失	BCEWithLogitsLoss
$\mathcal{L}_{cls}$	分类损失	BCEWithLogitsLoss（支持 Label Smoothing）

核心实现（`utils/loss.py`）：

```

1  class ComputeLoss:
2      def __call__(self, p, targets):
3          lcls = torch.zeros(1, device=self.device) # 分类损失
4          lbox = torch.zeros(1, device=self.device) # 框回归损失
5          lobj = torch.zeros(1, device=self.device) # 目标性损失
6          ...
7          lbox += (1.0 - iou).mean() # IoU 损失
8          lcls += self.BCEcls(pcls, t) # 分类 BCE
9          lobj += obji * self.balance[i] # 目标性 BCE
10         # 超参加权
11         lbox *= self.hyp["box"] # 0.0296 (调优) / 0.05 (baseline)
12         lobj *= self.hyp["obj"] # 0.301 / 1.0
13         lcls *= self.hyp["cls"] # 0.243 / 0.5
14         return (lbox + lobj + lcls) * bs, torch.cat((lbox, lobj, lcls)).detach()

```

调优实验中的损失权重（`hyp.voc_finetune.yaml`）相较 baseline 降低了 box/cls/obj 增益，避免微调初期梯度过大破坏预训练特征。

2.3 优化器与学习率设计

配置项	Baseline	调优（Finetuned）
优化器	SGD	SGD
初始学习率 <code>lr0</code>	0.01	0.003
最终学习率比例 <code>lrf</code>	0.01	0.12
动量 <code>momentum</code>	0.937	0.843
权重衰减	0.0005	0.00036
预热 epoch	3.0	2.0
学习率调度	线性余弦退火	线性余弦退火

训练命令示例（调优实验）：

```
1 python train.py \  
2   --data data/VOC.yaml \  
3   --weights weights/yolov5s.pt \  
4   --hyp data/hyps/hyp.voc_finetune.yaml \  
5   --epochs 50 --batch-size 32 --img 640 \  
6   --device 0 --workers 0 \  
7   --freeze 0 1 2 \           # 冻结 backbone 前 3 层  
8   --name voc_finetuned --patience 15
```

数据增强（调优超参）：

```
1 # data/hyps/hyp.voc_finetune.yaml（节选）  
2 mosaic: 0.8           # 略低于 baseline 的 1.0，减轻过拟合  
3 mixup: 0.05           # 轻度 mixup, baseline 为 0  
4 flipplr: 0.5  
5 hsv_h: 0.01041  
6 hsv_s: 0.54703  
7 scale: 0.75544
```

2.4 创新点与改进

本实验在标准 YOLOv5 微调流程基础上，做了以下**面向 VOC 域适配**的改进：

- 自定义微调超参文件 `hyp.voc_finetune.yaml`**
针对 COCO→VOC 迁移场景，将学习率降至 0.003，并补全 mosaic、mixup、HSV 等增强参数（原 `hyp.finetune.yaml` 不完整）。
- 部分 Backbone 冻结（`--freeze 0 1 2`）**
冻结网络前 3 层低层卷积，保留边缘/纹理等通用特征，仅训练高层语义与检测头，加速收敛并减少灾难性遗忘。
- 自动化实验分析脚本 `analyze_results.py`**
一键读取 `results.csv` 绘制 Baseline vs Finetuned 对比曲线，汇总 P/R/F1/mAP 指标表，输出 per-class 分析与 Markdown 报告。
- VOC 数据离线准备脚本 `scripts/prepare_voc.py`**

支持从 pjreddie 镜像下载 tar 包，自动解压并转换 XML 标注为 YOLO 格式，解决 GitHub 下载不稳定问题。

三、实验分析

3.1 数据集介绍

PASCAL VOC (Visual Object Classes) 是目标检测领域经典基准数据集，由牛津大学等机构发布。

属性	说明
类别数	20 类
训练集	VOC2007 trainval + VOC2012 trainval (约 16,551 张)
测试集	VOC2007 test (4,952 张, 12,032 个目标实例)
标注格式	原始 XML → 转换为 YOLO txt (<code>class cx cy w h</code> , 归一化)
类别示例	person, car, bus, cat, dog, bird, bottle, chair, ...

类别分布特点：

- `person` 实例最多 (4,528 个) , 占测试集目标总数约 37.6%
- 小目标类 (bottle、pottedplant、bird) 实例较少或目标尺寸小, 检测难度更高

数据目录结构：

```
1 exp5/datasets/VOC/
2 |— images/{train2012, val2012, train2007, val2007, test2007}/
3 |— labels/      (与 images 镜像对应)
```

3.2 实验设置

项目	Baseline	Finetuned
预训练权重	yolov5s.pt (COCO)	yolov5s.pt (COCO)
超参文件	hyp.scratch-low.yaml	hyp.voc_finetune.yaml
冻结层	无	backbone 层 0-2
训练 epoch	50	50
最优权重	runs/train/voc_baseline/weights/best.pt	runs/train/voc_finetuned/weights/best.pt

3.3 整体实验结果

在 VOC2007 test 集 (4,952 张) 上的评估结果如下 (数据来源: `runs/analysis/metrics_summary.csv`) :

模型	Precision	Recall	F1	mAP@0.5	mAP@0.5:0.95	检测准确率*
微调前 (epoch 0)	0.526	0.062	0.110	0.098	0.052	0.058
Baseline 微调	0.805	0.780	0.792	0.831	0.590	0.656
调优微调 (最终)	0.804	0.792	0.798	0.853	0.611	0.664

*检测准确率 = $P \times R / (P + R - P \times R)$

说明：COCO 预训练 `yolo5s.pt`（80 类）无法直接在 VOC（20 类）上运行 `val.py`，因此「微调前」基线采用 finetuned 训练第 0 epoch 的验证指标，反映检测头随机初始化后、尚未充分适配 VOC 的状态。

训练过程最优 epoch 指标（`results.csv` 末行）：

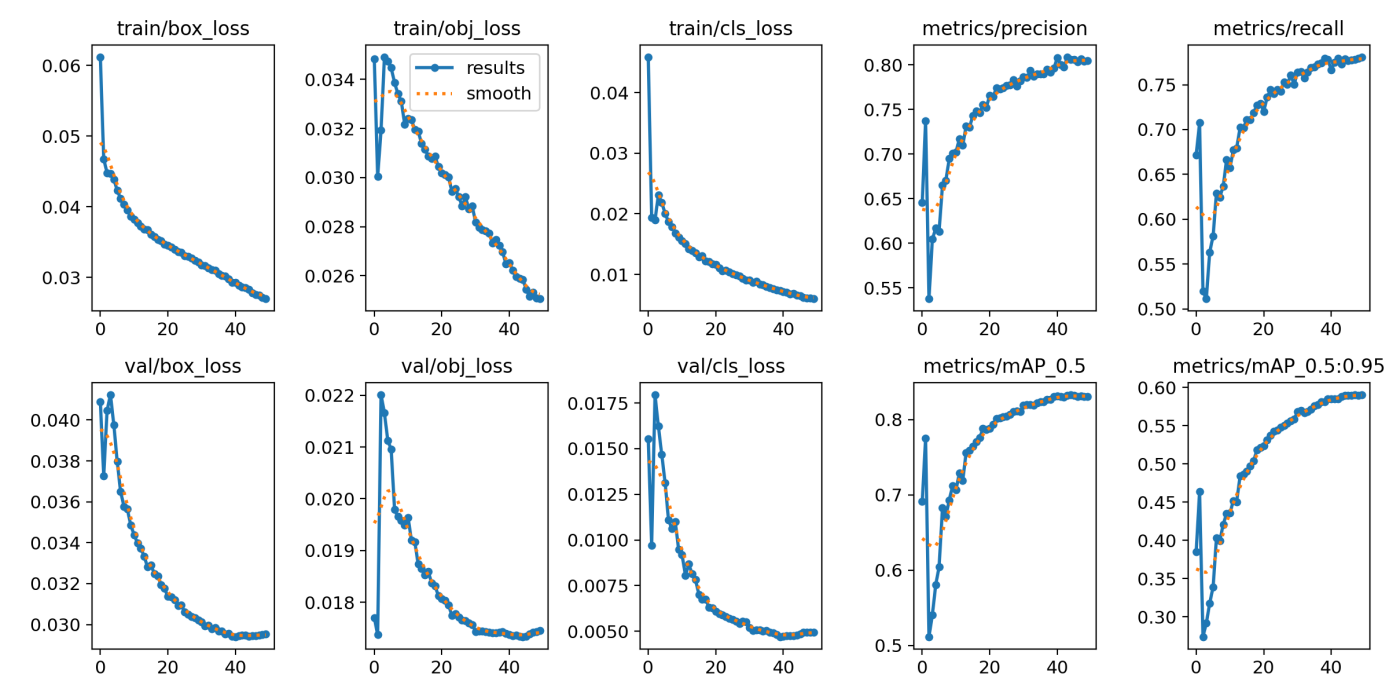
实验	Epoch	mAP@0.5	mAP@0.5:0.95	Precision	Recall
Baseline	49	0.831	0.590	0.805	0.781
Finetuned	49	0.854	0.612	0.805	0.793

关键结论：

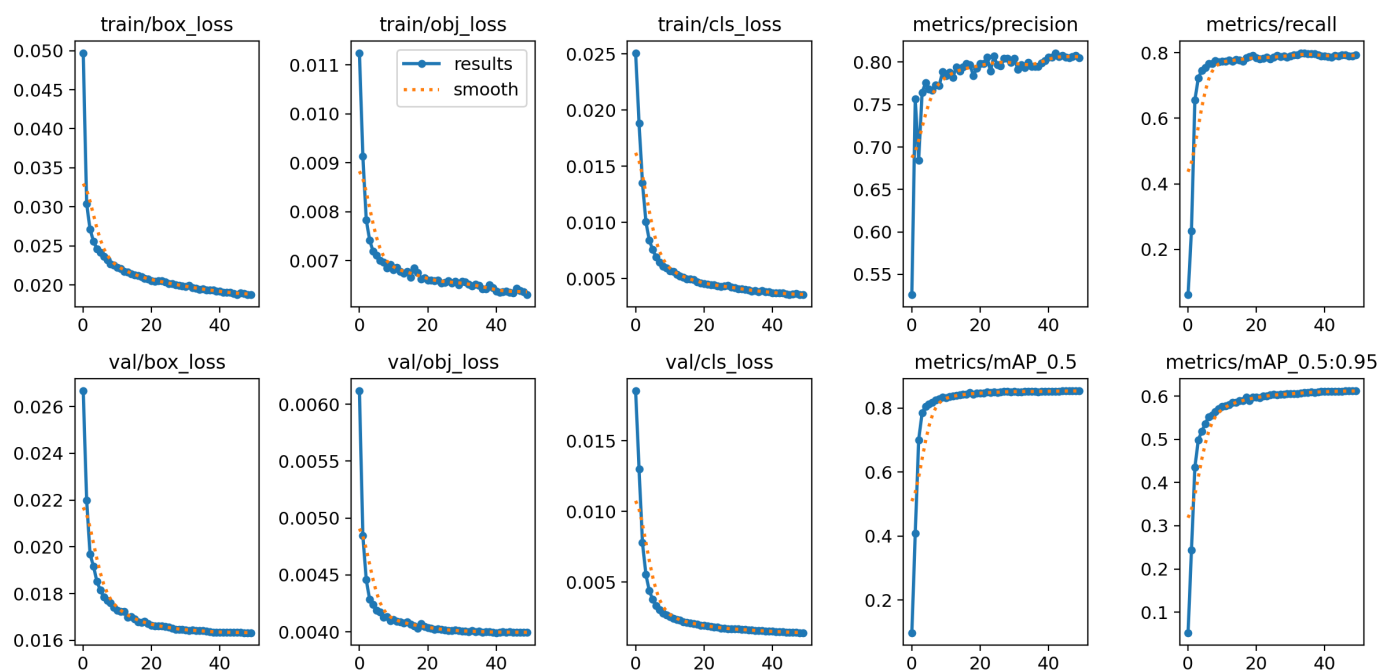
- 微调使 mAP@0.5 从 **0.098** → **0.853**，相对 COCO 权重直接用于 VOC 有 **+75.5%** 的绝对提升
- 调优超参相较 Baseline 进一步提升 mAP@0.5 **0.831** → **0.853**（**+2.2%**），Recall 从 0.780 提升至 0.792
- Finetuned 模型的 mAP@0.5:0.95 达到 **0.611**，高 IoU 阈值下定位更精确

3.4 训练曲线分析

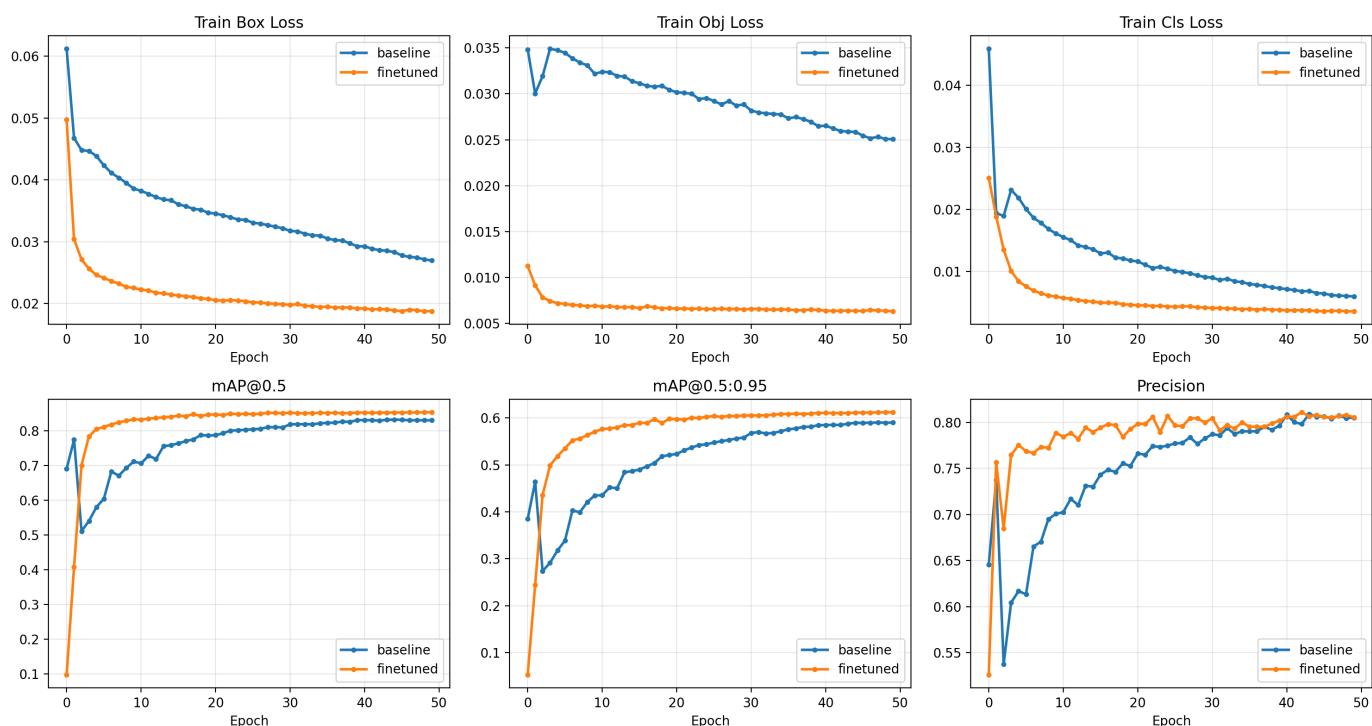
Baseline 训练曲线



Finetuned 训练曲线



两组实验对比



从对比图可以观察到：

- **损失下降**：两组实验 train box/obj/cls loss 均平稳下降；Finetuned 因冻结低层 + 低 lr，初期 loss 更低
- **mAP 上升**：Finetuned 的 mAP@0.5 收敛更快、最终值更高 (≈ 0.854 vs 0.831)
- **过拟合**：Baseline 与 Finetuned 的 val loss 与 train loss 差距均较小，未出现明显过拟合

3.5 各类别性能分析

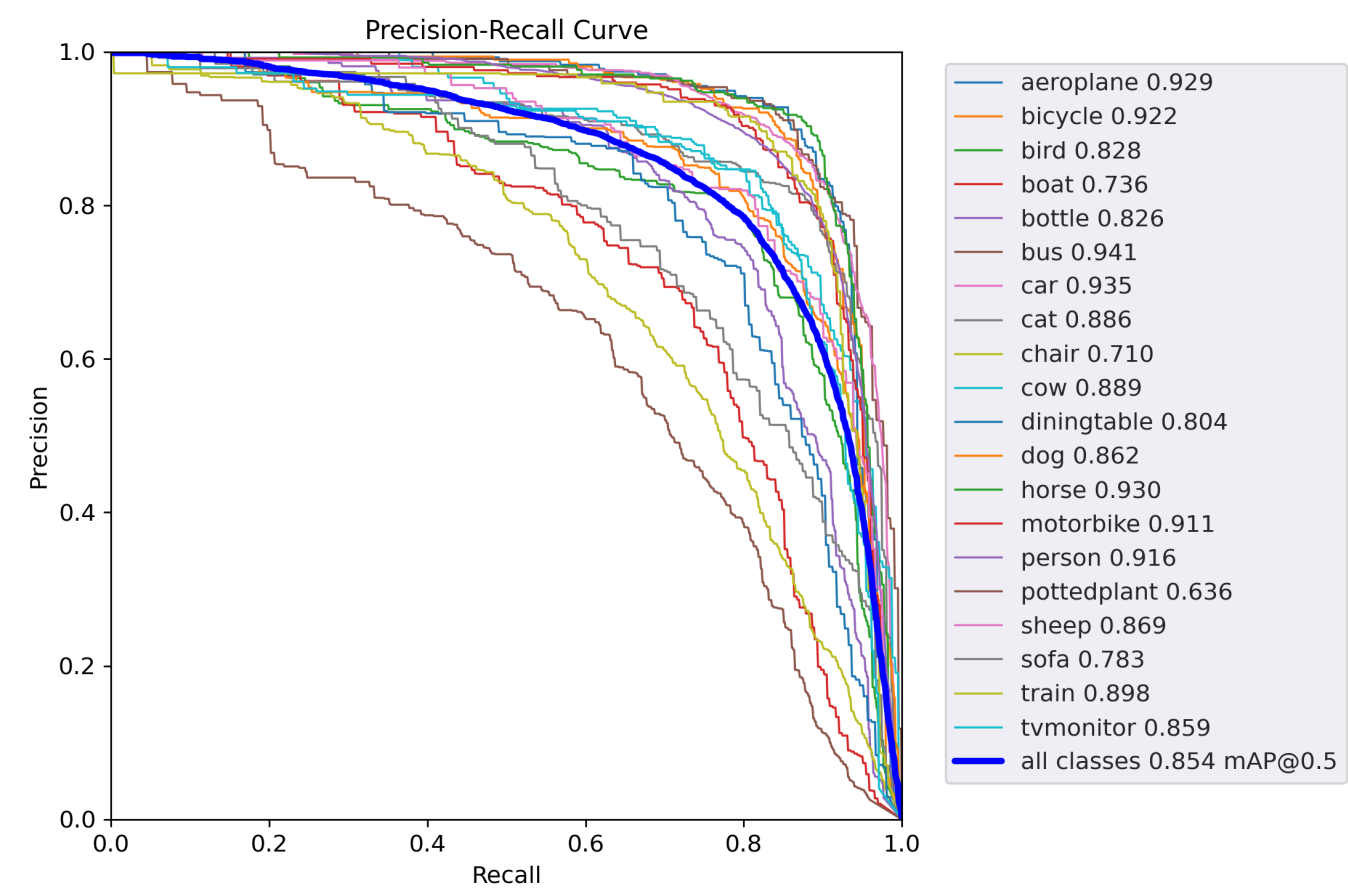
表现最好的 5 类 (Finetuned, mAP@0.5)：

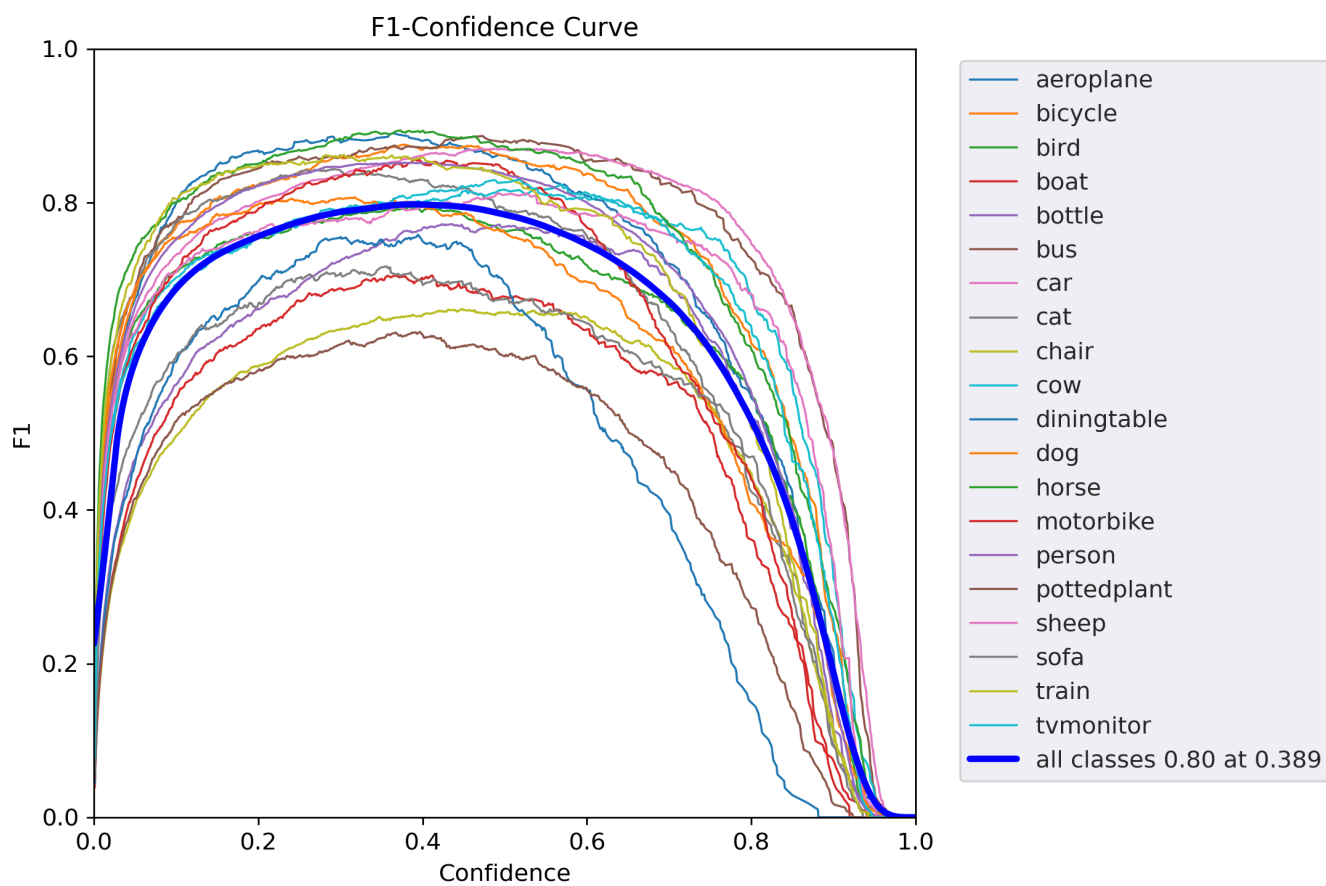
类别	Precision	Recall	mAP@0.5
bus	0.884	0.869	0.940
car	0.815	0.905	0.933
aeroplane	0.930	0.849	0.931
horse	0.905	0.876	0.927
bicycle	0.892	0.834	0.920

表现最弱的 5 类：

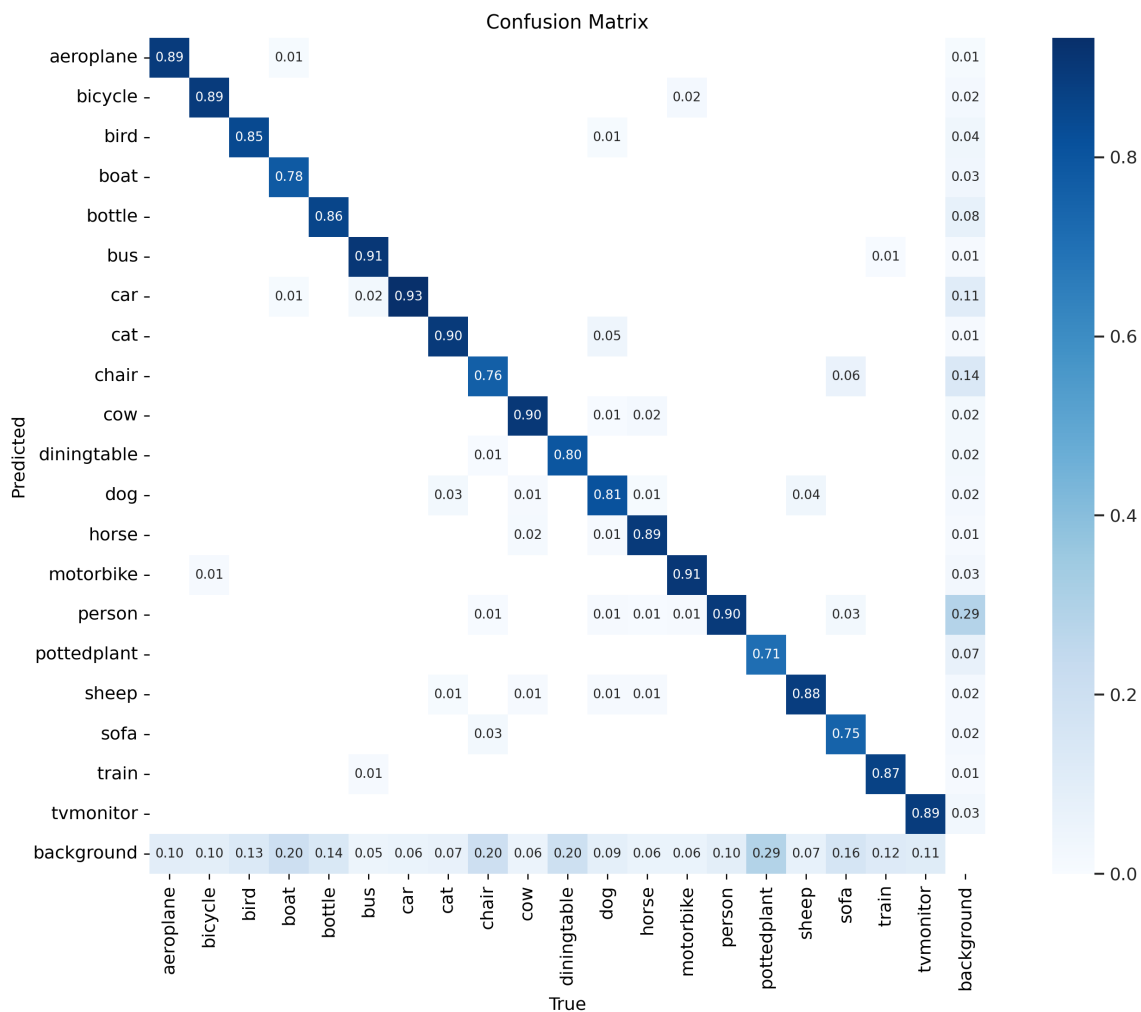
类别	Precision	Recall	mAP@0.5
pottedplant	0.638	0.596	0.638
chair	0.616	0.692	0.711
boat	0.703	0.667	0.717
sofa	0.767	0.648	0.789
diningtable	0.818	0.728	0.803

PR 曲线与 F1 曲线：





3.6 混淆矩阵与误检分析

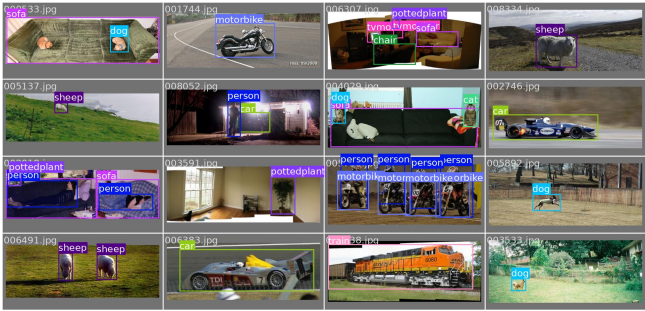
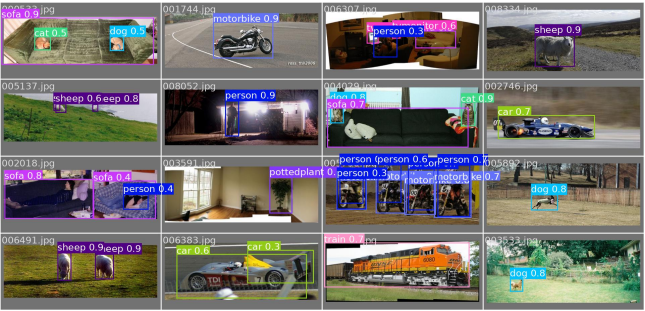


缺陷分析：

- 小目标检测不足**
pottedplant (0.638) 、boat (0.717) 等小目标 mAP 明显低于 bus/car 等大目标。小物体在 640×640 输入下占像素少，特征图分辨率不足导致漏检。
- 相似类别混淆**
混淆矩阵中 cat/dog、car/bus 等语义相近类别存在误分类，尤其 sofa 的 recall 仅 0.648，易与 chair 混淆。
- 类别不平衡**
person 实例占测试集 37.6%，模型对 person 的 mAP@0.5 达 0.916；而 pottedplant 仅 480 个实例，recall 偏低。
- 定位 vs 分类**
chair 的 recall (0.692) 高于 precision (0.616) ，说明模型能找到椅子但误检较多（背景误报）。

3.7 检测可视化

验证集预测对比 (Ground Truth vs Prediction)：

Ground Truth	Prediction
	

测试集单张检测示例（test2007/000001.jpg）：



全量 4,952 张测试集检测结果保存在 yolov5/runs/detect/voc_detect/。

四、总结

4.1 实验结论

1. 基于 **YOLOv5s + COCO** 预训练权重在 PASCAL VOC 上微调，可在 50 epoch 内将 mAP@0.5 从接近 0 提升至 **0.853**，验证了迁移学习在目标检测任务中的有效性。
2. 相较 Baseline 默认超参，本实验提出的 **低学习率 + 部分冻结 backbone + 轻度 mixup** 策略（hyp.voc_finetune.yaml）使 mAP@0.5 额外提升 **2.2%**，Recall 提升 **1.2%**。
3. 模型在 **大目标、外形规整类别**（bus、car、airplane）上表现优异（mAP@0.5 > 0.92），在 **小目标、形态多样类别**（pottedplant、chair、boat）上仍有改进空间。

4. 综合 Precision (0.804)、Recall (0.792)、F1 (0.798) 与 mAP@0.5 (0.853)，调优后的 **YOLOv5s-VOC** 模型 可作为 VOC 域目标检测的实用基线。

4.2 不足与展望

不足	改进方向
小目标 AP 偏低	增大输入分辨率（如 1280）、引入多尺度测试（TTA）
相似类别混淆	增加 hard example mining、类别平衡采样
Baseline 默认 lr 偏大	进一步网格搜索 lr、warmup、mosaic 等超参
仅测试 YOLOv5s	可对比 YOLOv5m/l 或 YOLOv8 等更大模型

4.3 产出物索引

类型	路径
最终推荐权重	<code>yolov5/runs/train/voc_finetuned/weights/best.pt</code>
训练日志	<code>yolov5/runs/train/voc_finetuned/results.csv</code>
指标汇总	<code>yolov5/runs/analysis/metrics_summary.csv</code>
对比曲线	<code>yolov5/runs/analysis/comparison.png</code>
验证结果	<code>yolov5/runs/val/voc_finetuned/</code>
检测可视化	<code>yolov5/runs/detect/voc_detect/</code>
复现手册	<code>exp5/readme.md</code>

实验环境：Conda `yolo` · PyTorch 2.6 · NVIDIA A800 · YOLOv5 v7.0